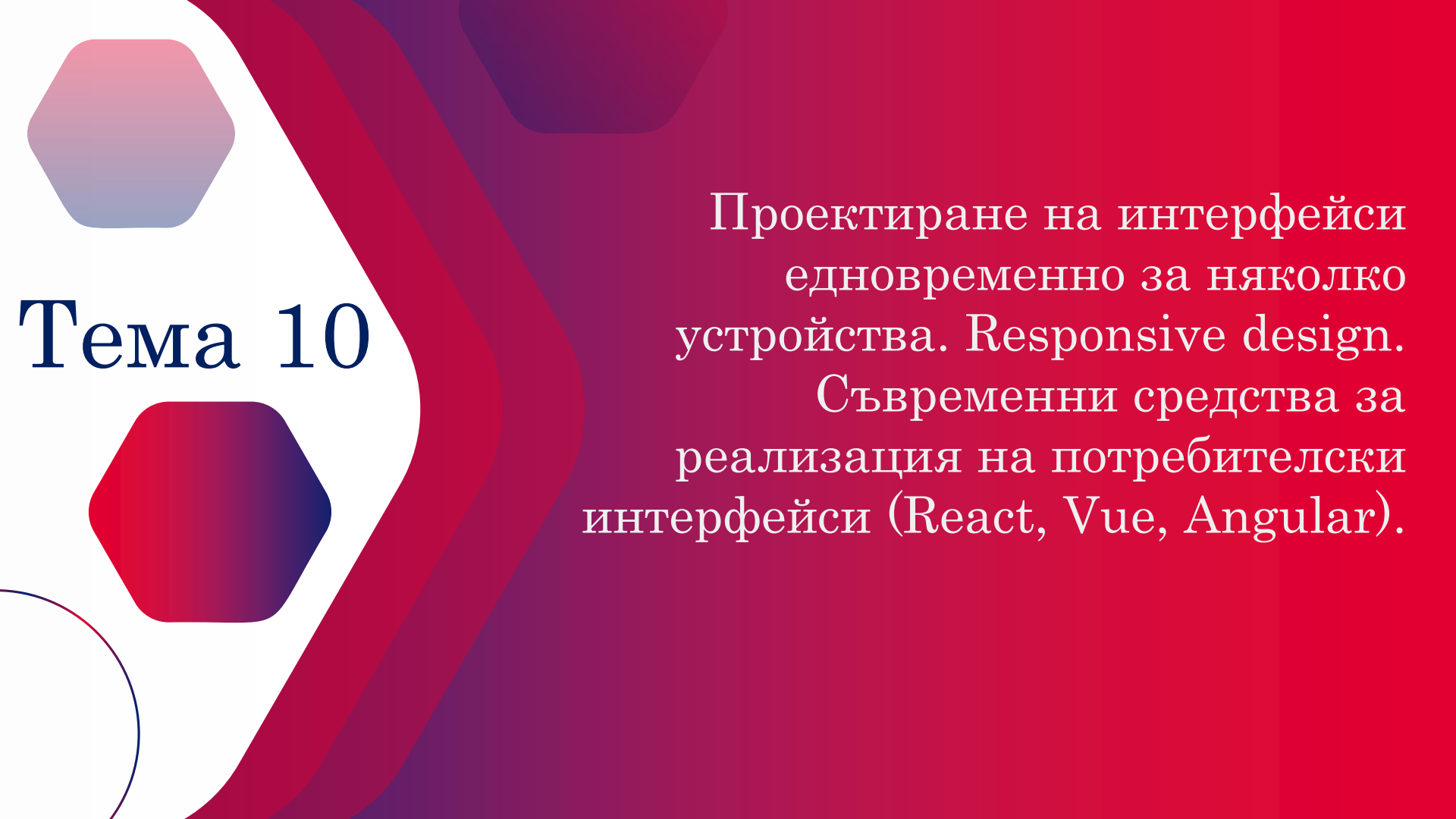




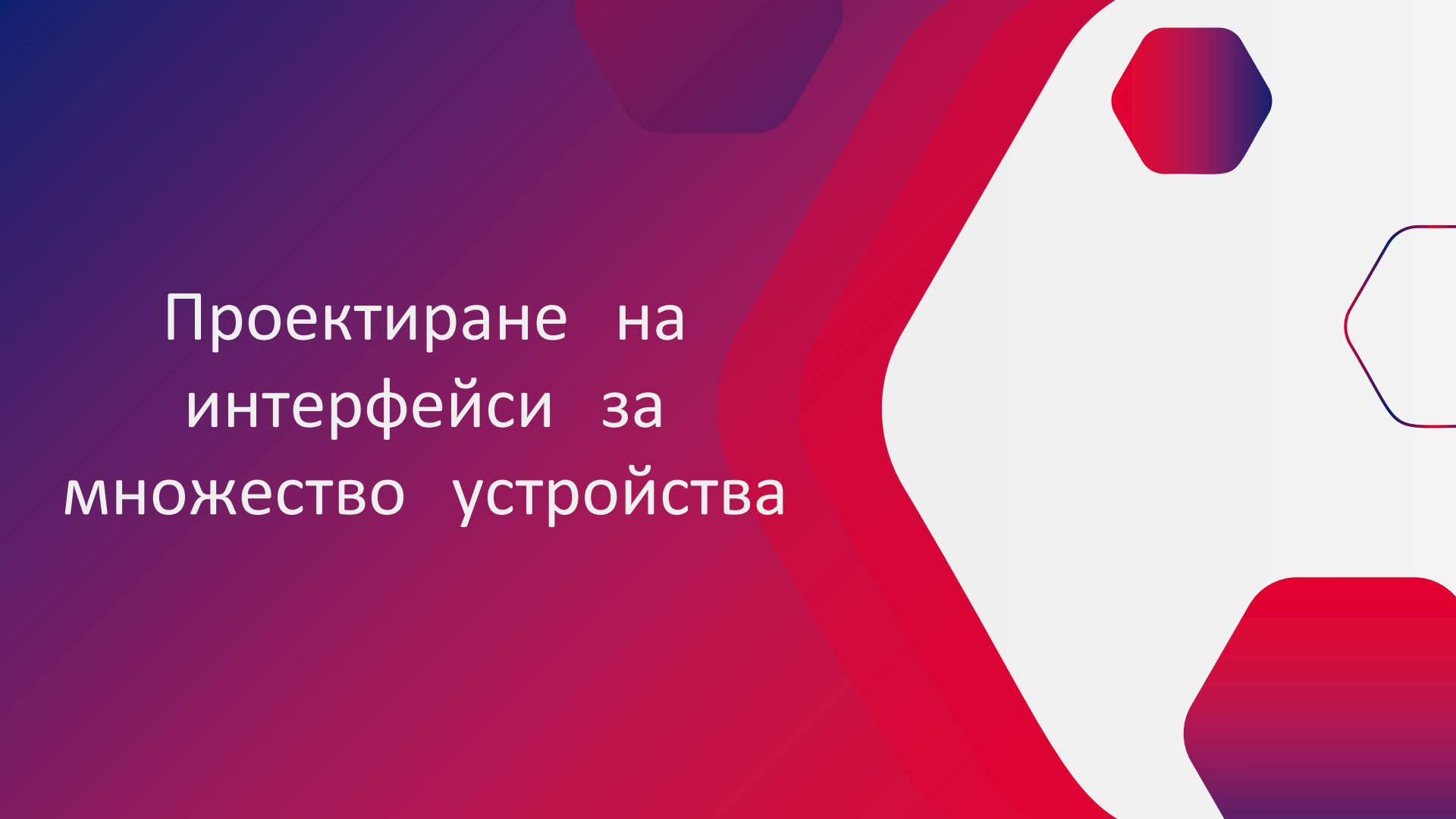
Проектиране на потребителски интерфейси

х.п. инж. Айше Ердинч Шабан



Тема 10

Проектиране на интерфейси
едновременно за няколко
устройства. Responsive design.
Съвременни средства за
реализация на потребителски
интерфейси (React, Vue, Angular).



Проектиране на
интерфейси за
множество устройства

Определение

Проектирането на интерфейси за множество устройства е процесът на създаване на потребителски интерфейси, които са оптимизирани за визуализация и функционалност на различни устройства, като например компютри, таблети и мобилни телефони. Този процес включва адаптиране на дизайна, структурата и функционалността на интерфейса, за да осигури консистентно и удобно изживяване на потребителите независимо от тяхното устройство и размер на екрана.



Кои са четирите С-та в дизайна на потребителския интерфейс (UX)?

- Съгласуваност
- Постоянство
- Контекст
- Допълващ се

First C of Four Cs in UX Design



CONSISTENCY



CONTINUITY



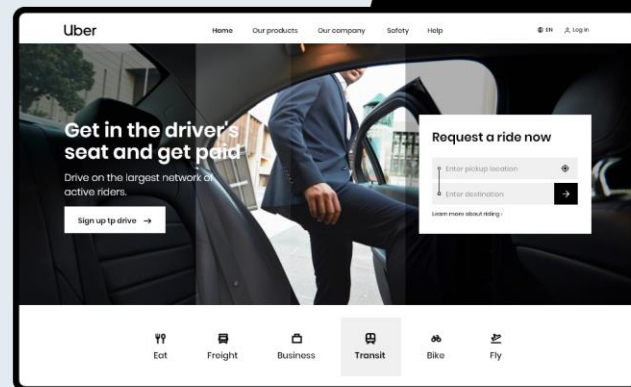
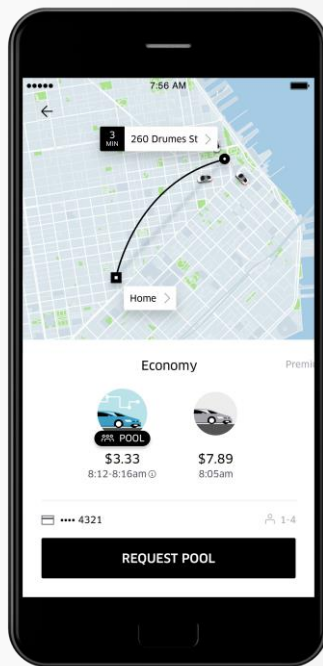
CONTEXT



COMPLEMENTARY

Основни принципи

- **Идентифициране на групите устройства за вашия продукт –**
 - ✓ Мобилни телефони
 - ✓ Таблети
 - ✓ Десктоп компютри
 - ✓ Смарт телевизори
 - ✓ Смарт часовници



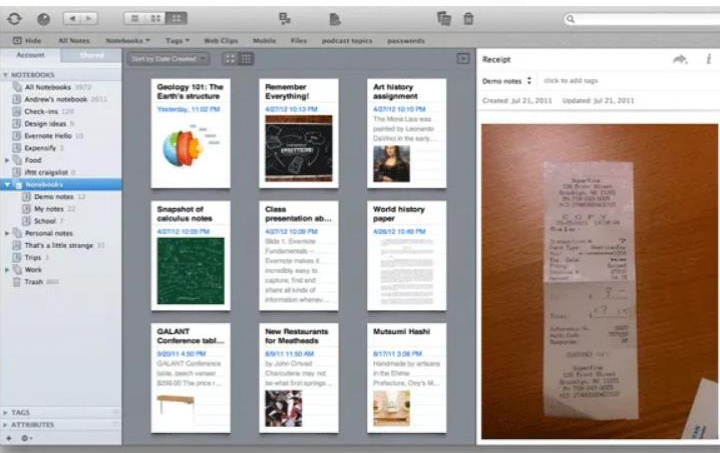
- **Идентифициране на основното потребителско изживяване**

“Кои са най-честите и важни задачи, които клиентите трябва да изпълнят?”

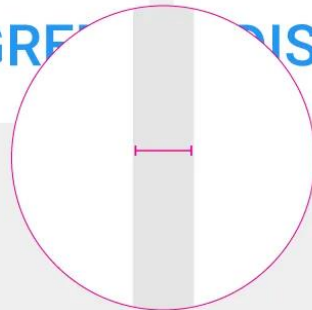
Основни принципи

- Адаптиране на изживяването за всеки контекст на употреба:

- ✓ Не всички функции имат смисъл на всички устройства
- ✓ Различните екрани позволяват използването на различни методи за въвеждане

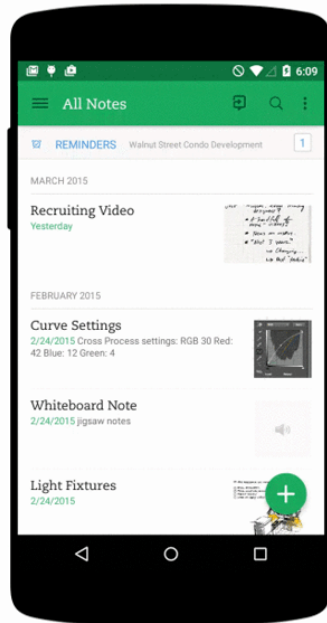


AGREE DISAGREE



Грешки при проектиране за устройства с докосване:

- Малки цели за докосване
- Поставяне на елементи твърде близо един до друг
- Използване на състояние "hover"



Основни принципи

- **Не забравяйте за големите екрани:**

- ✓ Не просто мащабирайте дизайна си, за да се побере на големите екрани.
- ✓ Уверете се, че изображенията не загубват качество, докато се увеличават за най-големите екрани.
- ✓ Обмислете специфичните особености на големи екрани.



- **Дизайн за най-малкия екран първо:**



Когато проектираме първо за десктоп, обикновено се сблъскваме с проблем: към продукта се добавят много функции, особено когато са включени множество заинтересовани страни

Основни принципи

- **Предоставете консистентно изживяване:**

- ✓ Приложението и неговото изживяване са подобни във всички размери на екрана

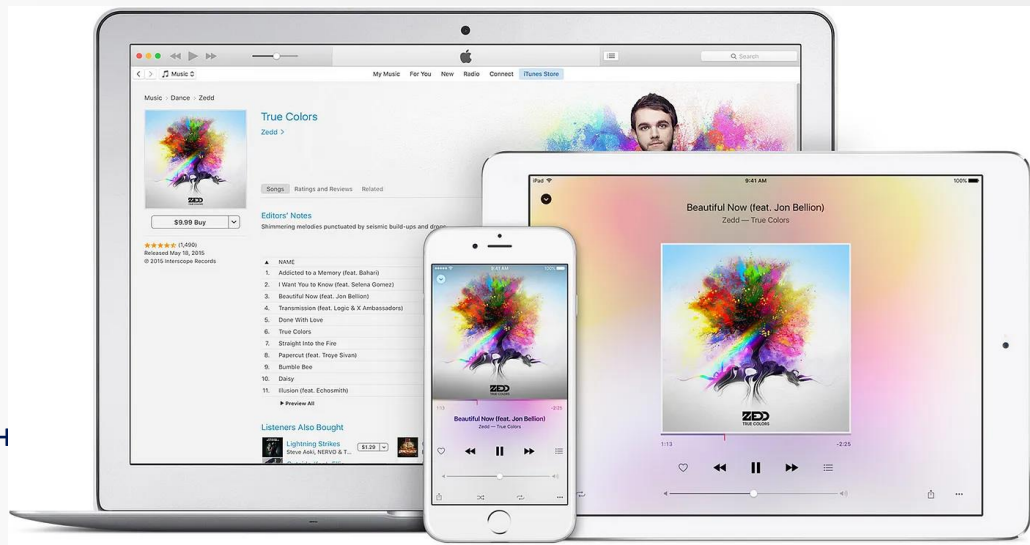


- **Тествайте вашия дизайн**

- ✓ Това, което работи в тестовите среди, не винаги задържа в реалния свят.

- **Създайте непрекъснато изживяване:**

- ✓ Потребителите не трябва да мислят за устройството, което използват, промените в средата или промените в контекста и могат да разчитат на отлична функционалност и лесна употреба независимо от устройството.



Responsive Design

The background features a gradient from dark purple on the left to white on the right. Large, thick, red and purple curved lines sweep across the right side. Several hexagonal shapes, some solid and some outlined, are scattered in the upper right quadrant.

Определение

Респонсивният дизайн е подход за уеб дизайн и разработка, който цели да осигури оптимално изживяване за потребителите на уебсайтове независимо от устройството, което използват за достъп.

ADAPTIVE

Generates templates which are optimized and unique for every device class



RESPONSIVE

Universal design which reflows across displays

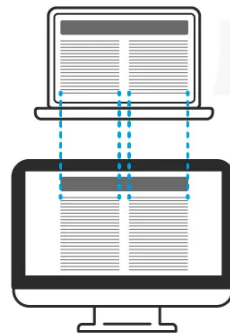


Той използва гъвкави мрежи, адаптиращи се изображения и CSS медиазапитвания, за да реагира и да се адаптира към различните размери на екраните и устройствата.

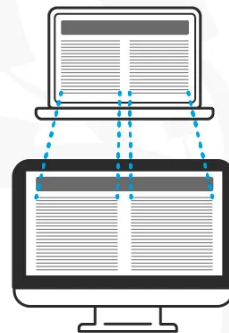
Основни принципи – Fluid Grid System

- Метод за организиране на съдържанието на уебсайта чрез използване на относителни, вместо абсолютни, размери за разпределение на елементите на страницата.
- Това позволява на елементите на уебсайта да се разпределят динамично и да се приспособяват към различните размери на екраните на устройствата, като се поддържа хармонично и балансирано изживяване за потребителите.

Fixed Version



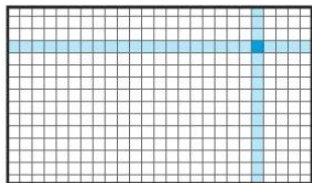
Fluid Version



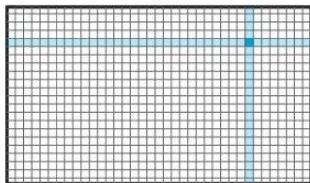
Често използвани единици:

- Проценти
- Viewport Width (vw) и Viewport Height (vh)
- min-width, max-width, min-height, max-height
- EM (em) и REM (rem)
- Viewport-Relative Lengths (vmin, vmax)

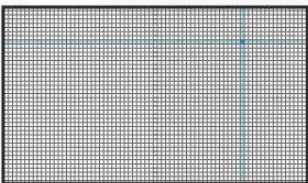
720p (HD)
1280 x 720 pixels



1080p (Full HD)
1920 x 1080 pixels

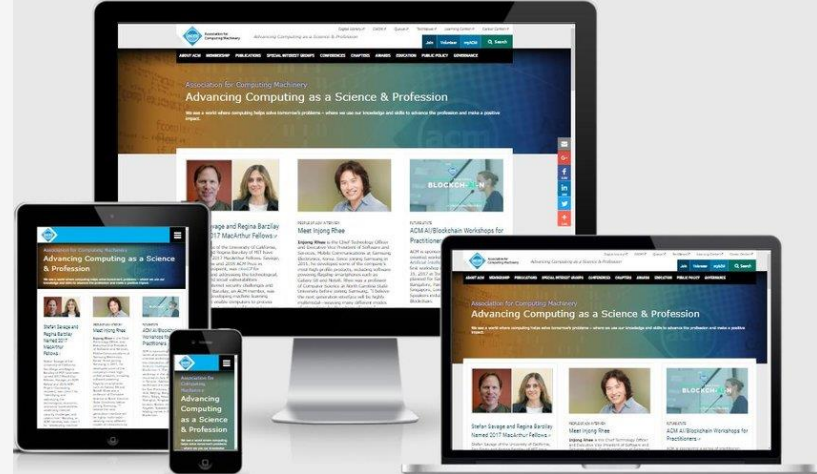
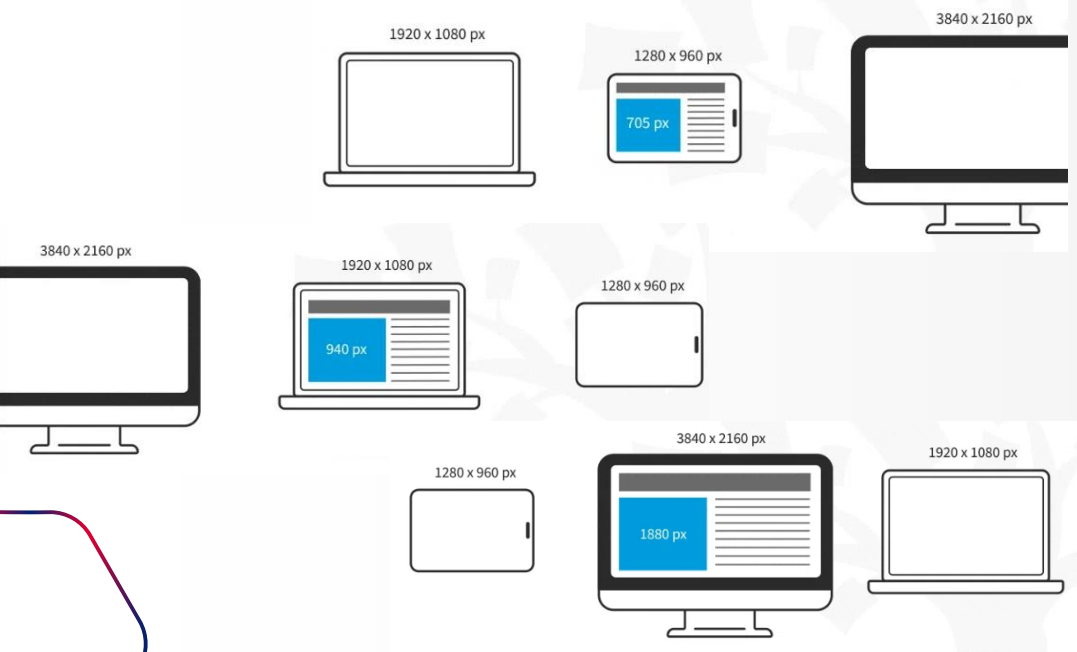


4K (Ultra HD)
3840 x 2160 pixels



Основни принципи – Fluid Image Use

- Вместо да задавате фиксирани размери за изображенията, можете да ги настроите да се скалират автоматично спрямо размера на техния контейнер.



- Гъвкавото използване на изображения не само осигурява оптимално изживяване за потребителите на различни устройства, но и помага за ускоряване на зареждането на уебсайта, като се намалява размерът на изображенията спрямо конкретните нужди на устройството.

Основни принципи – Media Queries

- Позволяват на уебсайтовете да реагират динамично на различните размери на екраните на устройствата, като променят стиловете и подреждането на съдържанието в зависимост от определени условия.
- Основната цел на медийните заявки е да предоставят инструкции на брауъра за това как да реагира на различни контексти на устройството, като например размерът на екрана, разделителната способност или ориентацията на устройството.
- breakpoints

Desktop



@media screen and
(min-width: 1024px)
{...}

Tablet



@media screen and
(min-width: 768px) and
(max-width: 1023px)
{...}

Smartphone

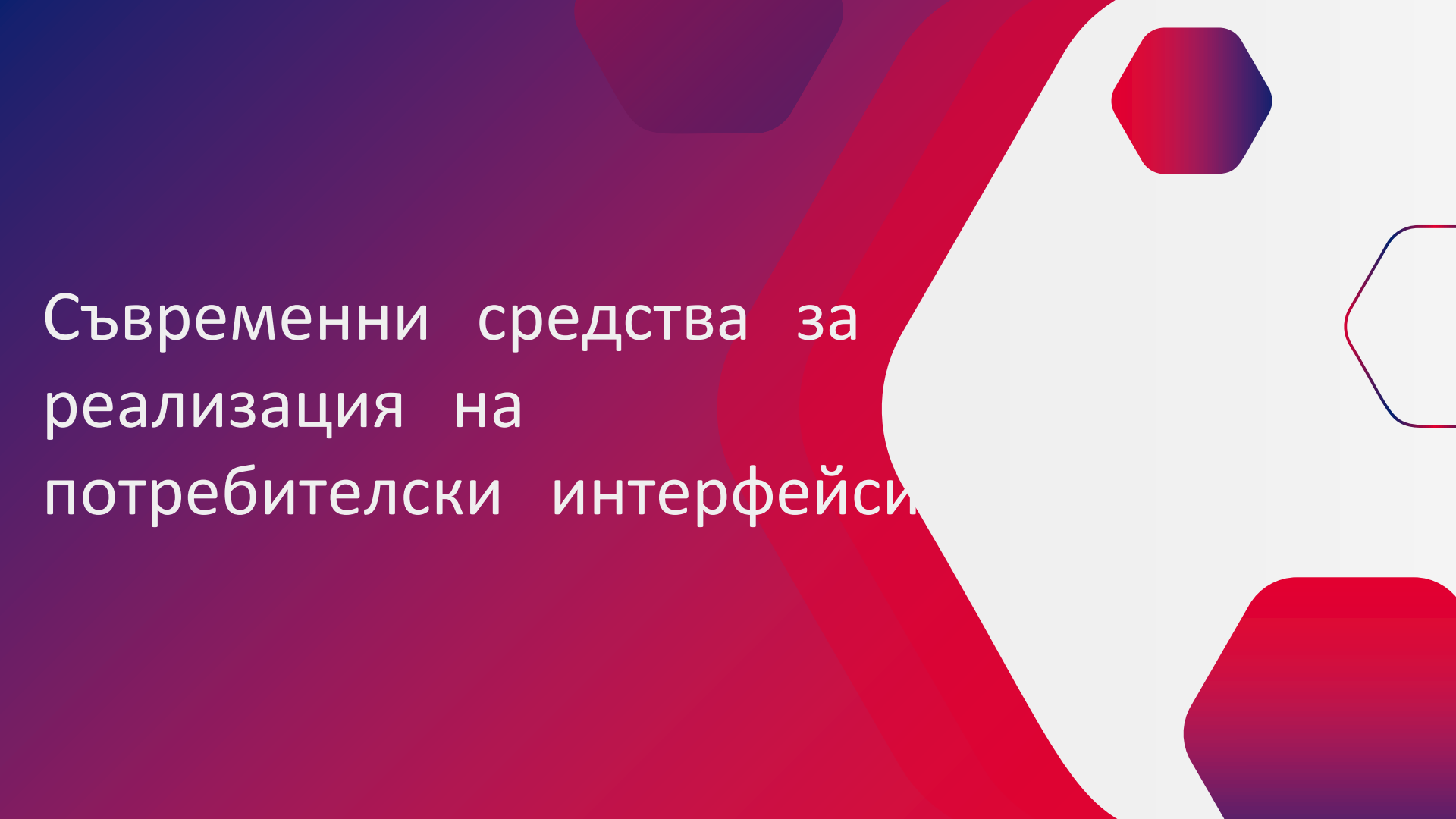


@media screen and
(max-width: 767px)
{...}

Най-добри практики

- Mobile-First Approach
- CSS Frameworks
- Респонсивни единици
- Мащабируема типография
- Медийни заявки
- Viewport Meta Tag
- Приоритизиране на съдържанието
- Тестване
- Оптимизация на производителността
- Потребителско изживяване

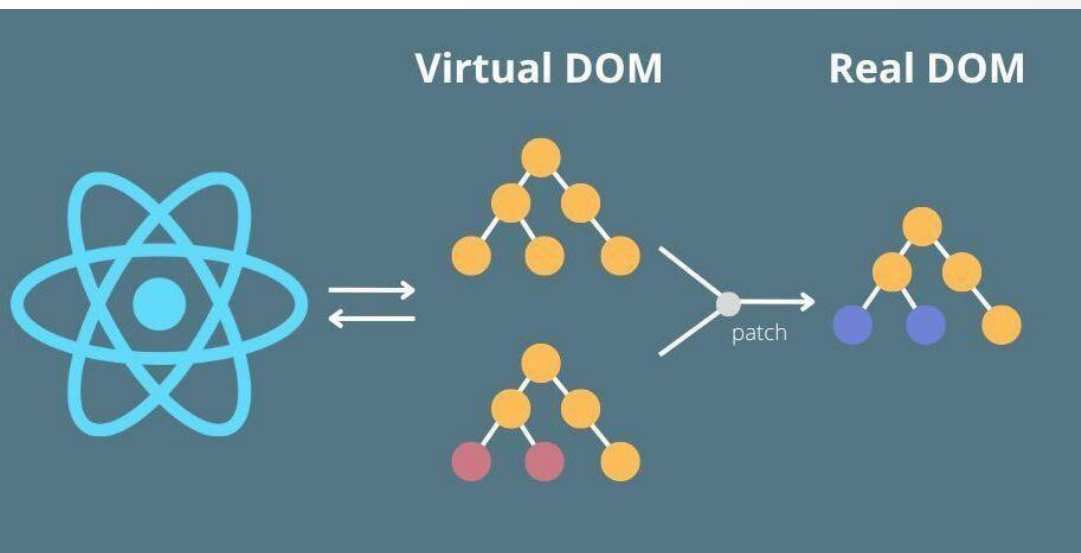




Съвременни средства за
реализация на
потребителски интерфейси

React – общ преглед и основни принципи

React (известен също като React.js или ReactJS) е JavaScript библиотека за изграждане на потребителски интерфейс, с която може да се програмират уебсайтове и мобилни софтуерни приложения



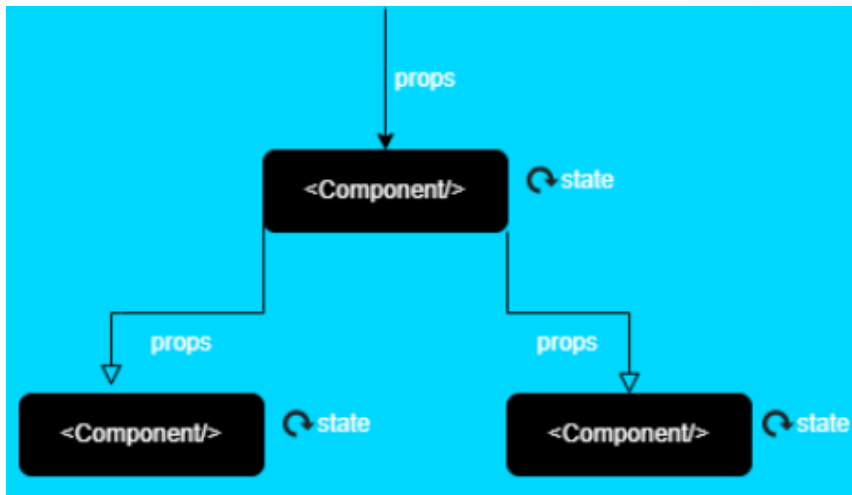
Virtual DOM

React използва виртуален DOM, който е абстракция над реалния DOM. Това позволява ефективно управление на промените в интерфейса и оптимизира производителността.



Състояние (State)

- React позволява деклариране и управление на състоянието на компонентите. Състоянието определя какво трябва да се показва в интерфейса и какво трябва да се случи при промяна.



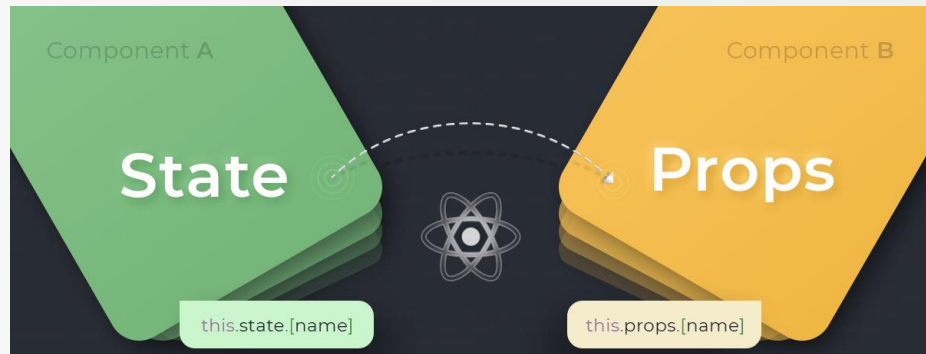
Пропс (Props)

- Пропс са параметри, които се предават на компоненти и могат да бъдат използвани за настройка и пренос на данни между компоненти.

React – основни принципи

Компоненти

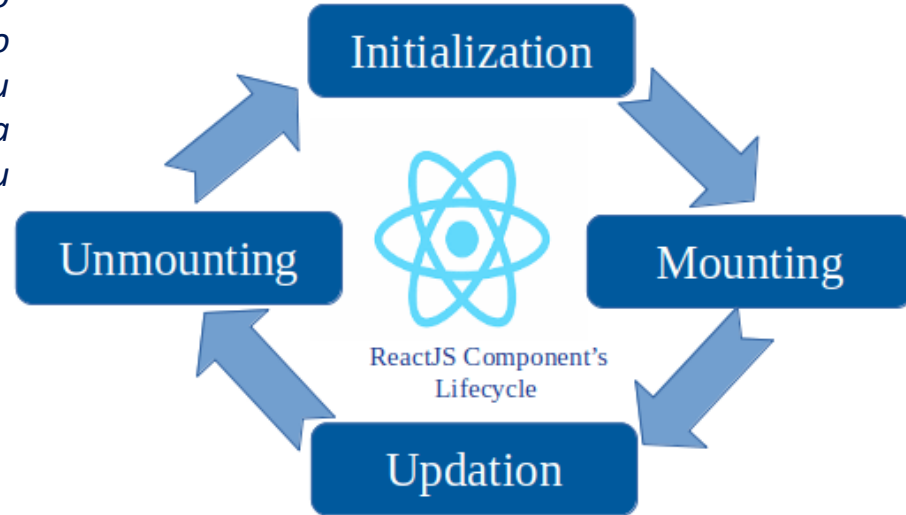
- Всеки интерфейсен елемент в React се представя като компонент.
- Компонентите могат да бъдат класови или функционални.
- Компонентите позволяват разделянето на интерфейса на по-малки, независими части, които могат да се използват и да се поддържат лесно.



React – основни принципи

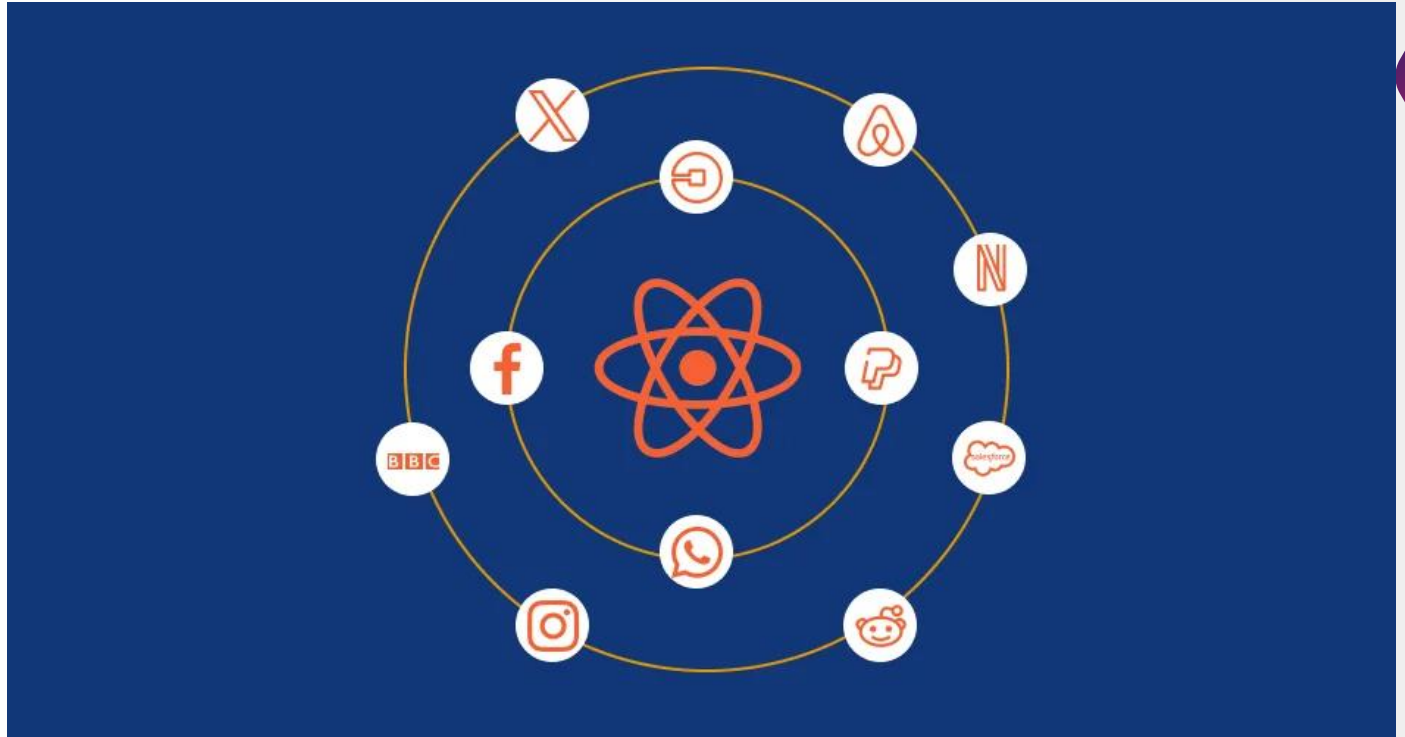
Жизнен цикъл (Lifecycle):

- Жизненият цикъл на компонентите в React представлява поредица от етапи, през които преминават компонентите от създаването им до тяхната демонтаж. React предоставя различни методи на жизнения цикъл, които позволяват на разработчиците да изпълняват код в определени точки от живота на компонента.
 - `componentDidMount()`
 - `componentDidUpdate(prevProps, prevState)`
 - `componentWillUnmount()`



React – примери

- *Facebook*
- *Instagram*
- *Netflix*
- *WhatsApp Web*
- *GitHub*



Vue – общ преглед

- *Vue.js е модерен JavaScript framework за създаване на потребителски интерфейси и един от най-популярните инструменти за уеб разработка в момента.*
- *Vue.js е лек, гъвкав и лесен за научаване framework за уеб разработка.*
- *Той се фокусира върху изграждането на потребителски интерфейси и е подходящ за различни видове приложения - от едностранни приложения (SPA) до по-традиционни мулти-странични приложения.*



Vue.js

Vue – основни принципи

- *Принцип на единичната отговорност*
- *Принцип на отворената/затворената функционалност*
- *Принцип на заместваемостта на Лисков*
- *Принцип на разделянето на интерфейса*
- *Принцип на инверсията на зависимостта*

Learning SOLID principles for better Vue 3 coding.



S - Single Responsibility
O - Open and Close
L - Liskov Substitution
I - Interface segregation
D - Dependency inversion

Vue – примеры

- *Facebook*
- *GitHub*
- *Netflix*
- *Adobe*



Angular – общ преглед

- *Angular е пълноценен уеб фреймуърк за разработване на съвременни уеб приложения и е известен със своите мощни възможности и широка общност от разработчици.*
- *Той предлага комплексен набор от функции, включително директиви, компоненти, услуги, рутване, форми и много други.*
- *Angular следва принципа на компонентна архитектура, където приложението е разделено на множество компоненти, които се състоят от HTML шаблони, TypeScript код и стилове CSS.*
- *Angular е силно разпространен уеб фреймуърк, разработен и поддържан от екипа на Google.*



Angular – основни принципи

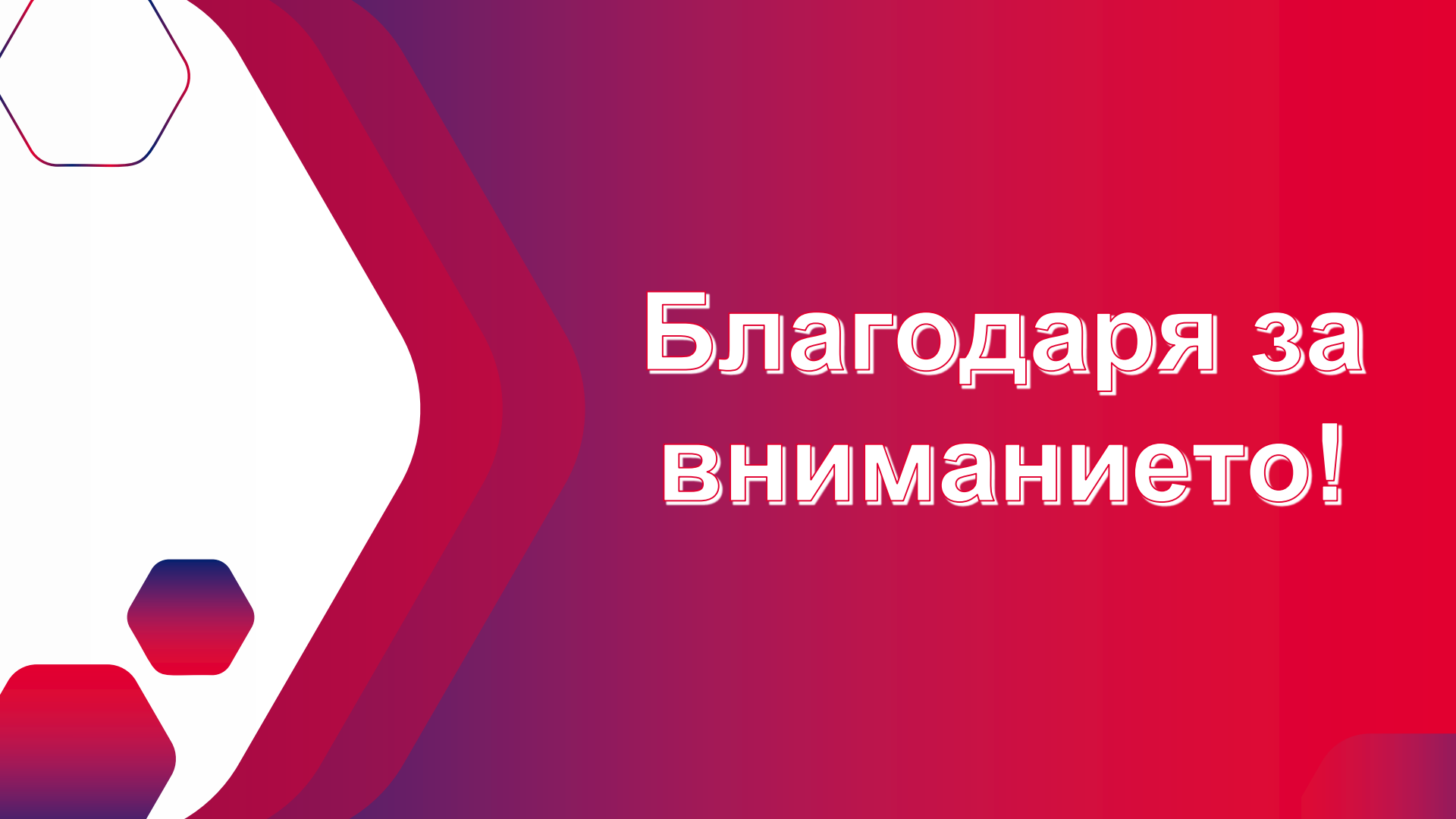
- *Принцип на единичната отговорност*
- *Принцип на отворената/затворената функционалност*
- *Принцип на заместваемостта на Лисков*
- *Принцип на разделянето на интерфейса*
- *Принцип на инверсията на зависимостта*



Angular – примери

- *Google*
- *YouTube TV*
- *Forbes*
- *Upwork*
- *PayPal*





**Благодаря за
вниманието!**